

ECOM EXPERTS

Ecom redis

Manual de usuario, tecnico y de desarrollo

ecom_redis 1.2.7 · 07/09/2026

Contenido

Configuración Inicial (Ejemplo Socket cPanel)

Configuración de Optimización TTFB

¿Cómo activarlo?

¿Qué sucede al activarlo?

Pestañas del Módulo

Soporte

Arquitectura de Caché

Optimizaciones Avanzadas (v1.3.0+)

Detalles de Implementación

Arquitectura Híbrida (LiteSpeed + Redis)

Sincronización

Flujo de Trabajo del Pre-Bootstrap

Gestión de Cookies

Desarrollo de Nuevos Hooks

Optimizaciones Pro de Redis

Estándares Gravity

Configuración Inicial (Ejemplo Socket cPanel)

Para entornos de hosting compartido (como cPanel con Sockets), utilice estos valores recomendados: -

Activar Redis: Sí (Enabled). - **Host / Socket:** `/tmp/redis.sock` (o la ruta que le facilite su proveedor). -

Puerto: `0` (Imprescindible usar 0 para conexiones por Socket). - **Contraseña:** Dejar en blanco (a menos que su proveedor le asigne una). - **Base de datos:** `0`. - **Prefijo:** `ps_` (o cualquier prefijo descriptivo).

Configuración de Optimización TTFB

La nueva pestaña **Optimización TTFB** permite activar el motor de "Pre-Bootstrap". Esta técnica es la más avanzada para mejorar la velocidad de carga (Time To First Byte).

¿Cómo activarlo?

1. Vaya a la pestaña **Optimización TTFB**.
2. Active el interruptor **Activar Pre-Bootstrap Engine**.
3. Pulse **Guardar**.

¿Qué sucede al activarlo?

El módulo modificará su archivo `.htaccess` para interceptar las visitas antes de que PrestaShop empiece a cargar archivos pesados. Si la página ya está en la memoria de Redis, se servirá instantáneamente (habitualmente en menos de 50ms).

Pestañas del Módulo

- **Dashboard:** Vista general del consumo de memoria y ratio de aciertos (Hits).
- **Page Cache (FPC):** Configuración de tiempos de vida (TTL) para cada tipo de página y gestión de limpieza manual.
- **Rendimiento Real:** Gráficas de comparación entre tiempos de carga con caché y sin caché.
- **Configuración Redis:** Datos de conexión a su servidor Redis (Host, Puerto, Puerto, etc).
- **PHP Opcache:** Herramientas para gestionar la memoria caché de scripts de PHP.
- **cPanel / WHM PECL:** Guía paso a paso para activar la extensión Redis en servidores con cPanel o administración WHM.
- **Exclusión de Contenido:** Permite definir rutas (URLs) o controladores específicos (como `cart`, `order`, `customer`) que NUNCA deben ser cacheados.
- **Optimización TTFB:** Control del motor de pre-carga rápida para un rendimiento extremo.

Soporte

Si tiene problemas con la activación del Pre-Bootstrap, asegúrese de que su servidor tiene permisos de escritura en el archivo `.htaccess` . Puede contactarnos en ecomyseo@gmail.com .

Documentación Técnica - Ecom Redis Manager

Arquitectura de Caché

El módulo utiliza una arquitectura de tres niveles para maximizar el rendimiento de PrestaShop:

1. Nivel 0: Pre-Bootstrap Engine (TTFB < 50ms)

- Implementado en `fpc_bootstrap.php`.
- Intercepta la petición desde `.htaccess` mediante `auto_prepend_file`.
- No carga el núcleo de PrestaShop.
- Utiliza scripts LUA atómicos para recuperación ultra-rápida.
- Detección de sesión mediante la cookie `ecom_no_cache`.
- Soporte para patrones de exclusión de URLs configurables.
- Validación de controladores excluidos para evitar servido dinámico erróneo.

2. Nivel 1: Full Page Cache (FPC)

- Implementado en `classes/EcomRedisPageCache.php`.
- Hook: `actionDispatcherBefore` (serve) y `actionOutputHTMLBefore` (store).
- Invalidación por etiquetas (Tags) para productos, categorías y CMS.

3. Nivel 2: Query / Object Cache

- Implementado en `classes/EcomRedisQueryCache.php`.
- Cachea resultados de bases de datos y objetos serializados.

Optimizaciones Avanzadas (v1.3.0+)

- **UNLINK (Background Delete):** Para borrar grandes lotes de claves, el módulo utiliza `UNLINK` en lugar de `DEL` (siempre que el servidor Redis sea v4.0+ y la extensión lo soporte), lo que permite que el borrado ocurra en segundo plano sin bloquear el hilo principal de Redis.
- **Protección contra Cache Stampede (Gen Lock):** Implementa un bloqueo de generación ("Gen Lock") para evitar que múltiples procesos PHP regeneren la misma página simultáneamente cuando expira la caché. Los procesos concurrentes esperan unos milisegundos a que el primer proceso almacene el resultado en Redis.
- **Serialización Igbinary:** Si la extensión `igbinary` está disponible, el módulo la utiliza para reducir drásticamente el tamaño de los datos en Redis y acelerar el proceso de serialización/deserialización.

Detalles de Implementación

- **LUA Scripts:** Se utilizan para reducir los viajes de ida y vuelta (round-trips) a Redis y asegurar atomicidad en el conteo de hits/misses.

- **Detección de Dispositivo:** Diferencia entre móvil (m) y escritorio (d) en la clave de caché.
- **Invalidación Selectiva:** Al actualizar un producto, se invalidan solo las páginas relacionadas (Home, Producto, Categorías asociadas).
- **Gestión de Carritos:** El sistema monitorea hooks de carrito (`actionCartSave` , `actionCartUpdateQuantityBefore` , `actionCartDelete`) para forzar la exclusión del FPC mediante una cookie de sesión robusta.

Arquitectura Híbrida (LiteSpeed + Redis)

El módulo permite una estrategia de caché en cascada: 1. **L1: LiteSpeed (Server Level):** Responde en < 10ms para el 99% de invitados. 2. **L2: Redis ULTRA-HIT (Bootstrap Level):** Responde si L1 falla, sin cargar PrestaShop. 3. **L3: MySQL Query Cache (Redis Level):** Reduce la carga de base de datos cuando la página es dinámica (ej: usuario logueado).

Sincronización

Cuando se activa la "Sincronización con LiteSpeed Cache", el módulo envía automáticamente cebeceras `X-LiteSpeed-Purge` y ejecuta el hook `actionPurgeLiteSpeedCache` cada vez que una entidad (producto, categoría o CMS) se actualiza. Esto garantiza que el contenido en el servidor LiteSpeed sea siempre idéntico al almacenado en Redis.

Guía Profesional de Desarrollo - Ecom Redis Manager

Flujo de Trabajo del Pre-Bootstrap

El archivo `fpc_bootstrap.php` es el punto de entrada para la optimización extrema del TTFB. Su flujo es: 1. **Validación de elegibilidad:** Solo peticiones GET, no AJAX, no estáticos. 2. **Configuración ligera:** Lee `config/settings.php` (generado por el módulo). 3. **Conexión Redis:** Usa persistencia si está configurada. 4. **Recuperación LUA:** Ejecuta script LUA para obtener el HTML y actualizar estadísticas en un solo paso. 5. **Output:** Si hay HIT, inyecta `window.ecom_ultra = true` y finaliza la ejecución con `exit`.

Gestión de Cookies

Para que el motor de pre-bootstrap sepa cuándo evitar la caché sin cargar PrestaShop, el módulo gestiona la cookie `ecom_no_cache`: - Se crea con `value=1` cuando hay productos en el carrito (`actionCartSave` , `actionCartUpdateQuantityBefore`). - Se crea al hacer login (`actionCustomerLoginAfter`). - Se elimina al vaciar el carrito (`actionCartDelete`) o hacer logout. - Utiliza parámetros de seguridad (Lax, Secure, HttpOnly) para compatibilidad con navegadores modernos.

Desarrollo de Nuevos Hooks

Si se añade un nuevo hook que modifique contenido cacheado, debe llamarse a `$this->invalidateRedisEntity($type, $id)` dentro del hook para mantener la consistencia de la caché.

Optimizaciones Pro de Redis

- **UNLINK:** En hooks de borrado masivo (`flushAll`), el sistema utiliza `UNLINK` si está disponible para evitar micro-cortes en el servicio durante limpiezas de caché grandes.
- **Igbinary:** Se utiliza para mejorar la velocidad de persistencia de datos complejos (Query Cache) reduciendo el tiempo de CPU en PrestaShop.
- **Stampede Lock:** Al desarrollar páginas de alto tráfico que consumen muchos recursos de DB, este mecanismo protege al servidor de picos repentinos de carga cuando expira la caché.

Estándares Gravity

- **Headers:** Todos los archivos PHP deben incluir la cabecera de licencia y autoría de Ecom Experts.
- **Seguridad:** Comprobación constante de `_PS_VERSION_`.
- **Compatibilidad:** Código compatible con versiones 8 y 9 de PrestaShop.